ZentroMart VoiceOps

Customer-support assistant — implementation and evaluation report

1. Project objective

This project develops a customer-support assistant for a synthetic e-commerce environment. Its intended workflow combines policy retrieval, catalog lookup, grounded response generation, order-verification gates, escalation tooling, and a Gnani-based speech interface.

2. Implemented components

- **Knowledge base:** Markdown policy documents are parsed into sections with metadata such as section IDs, document status, and effective dates.
- **Retrieval:** A lexical policy retriever selects relevant sections and can account for historical policy questions.
- **Catalog:** Product lookup reads the local CSV catalog and returns product attributes such as SKU, category, price, and warranty information.
- **Answer engine:** A standalone engine handles selected policy and catalog scenarios and returns evidence IDs for traceability.
- **Speech:** A Gnani-only STT-to-answer-to-TTS pipeline is configured. Actual end-to-end speech performance remains unverified until tested with a real recording and valid credentials.
- **Safety tooling:** An order-verification wrapper gates access to order details and limits verification attempts. Full integration and adversarial testing are still required.

3. Project files

The project includes a command-line entry point (run.py), batch audio processor (batch_process.py), source modules under src/, a requirements file, environment-variable template, and sample_outputs/ evaluation files. The .env.example contains a placeholder only; real credentials should stay in a local .env file excluded from version control.

4. Evaluation results

Metric	Result
Saved evaluation questions	52
Non-empty final answers	52
Empty final answers	0
Answer completeness	100.0%
Answerable questions in source dataset	47

Interpretation

The saved evaluation file contains 52 rows, and all 52 final-answer fields are non-empty. This supports a 100% answer-completeness result for the saved set only. It does not establish that every answer is factually correct, follows every policy condition, or matches a reference answer. Retrieval coverage figures, where available from the notebook, must be reported separately from answer accuracy.

Evaluation limitations

- No independently verified exact-match, semantic correctness, or policy-compliance score is claimed here.
- The saved CSV is treated as an existing artifact; this report does not rerun answer generation.
- Live Gnani transcription quality, synthesized audio quality, latency, and batch processing have not been benchmarked in this report.
- The answer engine currently covers selected scenarios rather than every possible support intent.

5. Reproducibility

Install dependencies from requirements.txt in the intended Python environment. Configure GNANI_API_KEY in a local .env file only when authorized credentials are available. Do not place credentials in source code, notebooks shared publicly, screenshots, or committed files.

Text-question smoke test

```
python run.py --question "What is the return window for electronics and accessories?"
```

Batch audio command

```
python batch_process.py --input-dir sample_outputs/customer_audio --output-dir
sample_outputs/batch
```

The batch command requires a directory containing real supported audio files and a working Gnani configuration. The dataset supplied for this project did not contain audio clips, so the command has not been validated against a real batch in this evaluation.

6. Recommended next validation steps

- Run a baseline-versus-final answer comparison using the reference answers and a documented scoring rubric.
- Check every answer against its required evidence, must-include conditions, and must-not-include conditions.
- Test current versus superseded policy precedence, category exceptions, festive-sale eligibility, and historical-policy dates.
- Exercise successful and failed order verification with synthetic test cases; verify no order data is returned before successful verification.
- With authorized Gnani credentials, record a short customer question and verify the complete STT → answer → TTS output files.
- Create a short demo video only after the real workflow has been run successfully; do not present a simulated run as live integration.

7. Conclusion

The project has a structured Python implementation, policy and catalog retrieval components, a saved 52-question answer artifact, and a configured speech integration path. Current evidence supports successful artifact creation and answer completeness, but additional correctness, safety, and live speech testing is needed before claiming end-to-end production readiness.